

MODUL LENGKAP BELAJAR SQL

Dari Dasar Hingga Advanced

Penjelasan Mendalam · Contoh Nyata · Latihan & Kunci Jawaban

■ BASIC

■ INTERMEDIATE

■ ADVANCED

LEVEL	TOPIK	JUMLAH
■ Basic	SELECT, WHERE, INSERT, UPDATE, DELETE, ORDER BY, LIMIT & OFFSET	7 Topik
■ Intermediate	COUNT, GROUP BY, CREATE TABLE, PRIMARY KEY, DISTINCT, LIKE, IN, BETWEEN, ALIAS, SUM & AVG, HAVING, JOIN, SUBQUERY, INDEX	14 Topik
■ Advanced	UNION, CASE, VIEW, ALTER TABLE, NOT NULL & DEFAULT, TRANSACTION, NORMALISASI, WINDOW FUNCTION, CTE, STORED PROCEDURE, TRIGGER, QUERY OPTIMIZATION	12 Topik

Dataset standar digunakan di seluruh modul agar kamu bisa fokus pada perbedaan setiap perintah.

Dataset Standar — Tabel users

id	nama	umur	kota
1	Budi	20	Bandung
2	Andi	25	Jakarta
3	Rina	22	Bandung
4	Sari	30	Surabaya

Dataset Standar — Tabel orders

id	user_id	produk	harga
1	1	Laptop	12.000.000
2	2	Mouse	150.000
3	1	Keyboard	450.000
4	3	Monitor	2.500.000

■ LEVEL BASIC — Fondasi SQL

1. SELECT — Mengambil Data

■ Penjelasan Mendalam

SELECT adalah perintah paling fundamental dalam SQL. Fungsinya adalah **membaca dan menampilkan data** dari satu atau lebih tabel. SQL bersifat **deklaratif** — kita hanya menyebutkan *apa* yang kita mau, bukan *bagaimana* cara mengambilnya. Database engine yang mengurus prosesnya.

■ ■ Kenapa jangan pakai `SELECT *` di production? Mengambil semua kolom memboroskan bandwidth, memperlambat query, dan bisa membocorkan kolom sensitif (seperti `password_hash`). Selalu sebut nama kolom secara eksplisit.

■ Contoh Query

Contoh 1 — Ambil semua kolom

```
SELECT * FROM users;
```

id	nama	umur	kota
1	Budi	20	Bandung
2	Andi	25	Jakarta
3	Rina	22	Bandung
4	Sari	30	Surabaya

Contoh 2 — Ambil kolom tertentu

```
SELECT nama, kota FROM users;
```

nama	kota
Budi	Bandung
Andi	Jakarta
Rina	Bandung
Sari	Surabaya

Contoh 3 — SELECT dengan kalkulasi

```
SELECT nama, umur, umur + 5 AS umur_5_tahun_lagi FROM users;
```

nama	umur	umur_5_tahun_lagi
Budi	20	25
Andi	25	30
Rina	22	27
Sari	30	35

■ LATIHAN

Pilihan Ganda:

1. Apa fungsi utama perintah `SELECT` dalam SQL?

- A. Menghapus data dari tabel
- B. Mengambil dan menampilkan data dari tabel
- C. Mengubah struktur tabel
- D. Menambahkan baris baru ke tabel

2. Query mana yang HANYA menampilkan kolom nama dari tabel users?

- A. SELECT users FROM nama;
- B. SELECT * FROM users;
- C. SELECT nama FROM users;
- D. FROM users SELECT nama

3. Apa arti tanda * pada SELECT * FROM users?

- A. Pilih data secara acak
- B. Lakukan perkalian pada semua data
- C. Ambil semua kolom dari tabel
- D. Ambil hanya kolom pertama

Esai:

1. Jelaskan perbedaan antara SELECT * dan SELECT nama, umur dari sisi performa dan keamanan data di aplikasi nyata!
2. Tuliskan query SQL untuk menampilkan kolom nama dan hasil perkalian umur dengan 2, dengan nama kolom hasil berupa "dua_kali_umur". Jelaskan tiap bagian query tersebut!

2. WHERE — Filter / Menyaring Data

■ Penjelasan Mendalam

WHERE adalah klausa yang diletakkan setelah nama tabel untuk **memfilter baris** mana saja yang akan dikembalikan. Database mengecek setiap baris satu per satu; hanya baris yang memenuhi kondisi WHERE yang akan masuk ke hasil query. Tanpa WHERE, query akan mengambil **semua baris** dalam tabel.

■ Operator Perbandingan

Operator	Arti	Contoh
=	Sama dengan	WHERE kota = 'Bandung'
!= / <>	Tidak sama dengan	WHERE kota != 'Jakarta'
>	Lebih besar	WHERE umur > 20
<	Lebih kecil	WHERE umur < 30
AND	Kedua kondisi harus terpenuhi	WHERE umur > 20 AND kota = 'Bandung'
OR	Salah satu kondisi terpenuhi	WHERE kota = 'Bandung' OR kota = 'Jakarta'
NOT	Membalik kondisi	WHERE NOT kota = 'Surabaya'

■ Contoh Query

Contoh 1 — Filter angka

```
SELECT * FROM users WHERE umur > 22;
```

id	nama	umur	kota
2	Andi	25	Jakarta
4	Sari	30	Surabaya

Budi (20) dan Rina (22) tidak muncul karena tidak memenuhi syarat umur > 22.

Contoh 2 — Filter teks

```
SELECT * FROM users WHERE kota = 'Bandung';
```

id	nama	umur	kota
1	Budi	20	Bandung
3	Rina	22	Bandung

Contoh 3 — Kombinasi AND

```
SELECT * FROM users WHERE umur > 20 AND kota = 'Bandung';
```

id	nama	umur	kota
3	Rina	22	Bandung

Hanya Rina yang memenuhi kedua syarat sekaligus. Budi gagal karena umur = 20 (tidak > 20).

■ LATIHAN

Pilihan Ganda:

1. Apa fungsi klausa WHERE dalam SQL?
 - A. Mengurutkan hasil query
 - B. Mengelompokkan data berdasarkan kolom
 - C. Memfilter baris berdasarkan kondisi tertentu
 - D. Membatasi jumlah kolom yang ditampilkan
2. Query mana yang mengambil semua user dari kota Jakarta?
 - A. `SELECT * FROM users WHERE kota = 'Jakarta';`
 - B. `SELECT * FROM users FILTER kota = 'Jakarta';`
 - C. `SELECT kota FROM users WHERE = 'Jakarta';`
 - D. `SELECT * FROM users kota = 'Jakarta';`
3. Operator mana yang digunakan untuk kondisi 'tidak sama dengan'?
 - A. `==`
 - B. `=`
 - C. `>=`
 - D. `!=`

Esai:

1. Jelaskan perbedaan antara operator AND dan OR dalam klausa WHERE, dan berikan masing-masing satu contoh query menggunakan dataset di atas!
2. Mengapa sangat penting menggunakan WHERE saat melakukan perintah DELETE atau UPDATE? Apa yang terjadi jika kita lupa menambahkan WHERE?

3. INSERT — Menambah Data

■ Penjelasan Mendalam

INSERT INTO digunakan untuk **menambahkan satu atau lebih baris baru** ke dalam tabel. Data yang dimasukkan harus sesuai dengan tipe data kolom. Jika ada kolom dengan constraint **NOT NULL**, kolom tersebut wajib diisi.

Contoh 1 — Insert satu baris

```
INSERT INTO users (id, nama, umur, kota)
VALUES (5, 'Doni', 28, 'Jakarta');
```

Contoh 2 — Insert banyak baris sekaligus (lebih efisien)

```
INSERT INTO users (id, nama, umur, kota)
VALUES
(7, 'Tono', 27, 'Semarang'),
(8, 'Wati', 24, 'Yogyakarta');
```

■■ INSERT banyak baris sekaligus jauh lebih cepat daripada INSERT satu per satu karena hanya membutuhkan satu round-trip ke database.

■ LATIHAN

Pilihan Ganda:

1. Perintah SQL mana yang digunakan untuk menambahkan data baru ke tabel?
 - A. ADD INTO
 - B. INSERT INTO
 - C. UPDATE INTO
 - D. CREATE INTO
2. Jika tabel users memiliki kolom id sebagai PRIMARY KEY, apa yang terjadi jika kita INSERT dengan id yang sudah ada?
 - A. Data lama otomatis tertimpa
 - B. Data baru ditambahkan dengan ID baru
 - C. Query gagal dan menghasilkan error
 - D. Query berhasil dan menghasilkan dua baris dengan ID sama
3. Manakah sintaks INSERT yang benar?
 - A. INSERT users VALUES (5, 'Doni', 28, 'Jakarta');
 - B. INSERT INTO users VALUES 5, 'Doni', 28, 'Jakarta';
 - C. INSERT INTO users (id, nama, umur, kota) VALUES (5, 'Doni', 28, 'Jakarta');
 - D. INTO users INSERT (id, nama) VALUES (5, 'Doni');

Esai:

1. Jelaskan keuntungan melakukan INSERT banyak baris sekaligus dibandingkan INSERT satu per satu dalam hal performa database!
2. Apa yang dimaksud dengan nilai NULL pada kolom yang tidak diisi saat INSERT? Kapan hal ini diperbolehkan dan kapan tidak?

4. UPDATE — Mengubah Data

■ Penjelasan Mendalam

UPDATE digunakan untuk **memodifikasi nilai** pada baris yang sudah ada di tabel. Kamu bisa mengubah satu kolom saja atau beberapa kolom sekaligus dalam satu query.

■ **SANGAT PENTING:** Selalu sertakan WHERE saat UPDATE! Tanpa WHERE, SEMUA baris di tabel akan berubah.

Contoh 1 — Update satu kolom

```
UPDATE users SET umur = 26 WHERE id = 2;
```

Contoh 2 — Update beberapa kolom sekaligus

```
UPDATE users SET kota = 'Bekasi', umur = 21 WHERE id = 1;
```

Contoh 3 — Update berdasarkan kondisi non-ID

```
UPDATE users SET kota = 'Jakarta' WHERE kota = 'Bandung';
```

Sesudah: Semua user yang tinggal di Bandung (Budi dan Rina) sekarang pindah ke Jakarta.

■ LATIHAN

Pilihan Ganda:

1. Apa yang terjadi jika kita menjalankan `UPDATE users SET umur = 99;` tanpa `WHERE`?
 - A. Hanya baris pertama yang berubah
 - B. Query gagal karena tidak ada `WHERE`
 - C. Semua baris di tabel akan memiliki umur = 99
 - D. Tidak ada yang berubah
2. Query mana yang benar untuk mengubah kota Rina menjadi 'Yogyakarta'?
 - A. `CHANGE users SET kota = 'Yogyakarta' WHERE nama = 'Rina';`
 - B. `UPDATE users SET kota = 'Yogyakarta' WHERE nama = 'Rina';`
 - C. `UPDATE kota = 'Yogyakarta' FROM users WHERE nama = 'Rina';`
 - D. `SET kota = 'Yogyakarta' IN users WHERE nama = 'Rina';`
3. Bagaimana cara mengubah dua kolom sekaligus dalam satu perintah `UPDATE`?
 - A. `UPDATE users SET umur = 25; SET kota = 'Jakarta' WHERE id = 1;`
 - B. `UPDATE users SET (umur = 25, kota = 'Jakarta') WHERE id = 1;`
 - C. `UPDATE users SET umur = 25, kota = 'Jakarta' WHERE id = 1;`
 - D. `UPDATE users umur = 25, kota = 'Jakarta' WHERE id = 1;`

Esai:

1. Mengapa sangat dianjurkan untuk selalu menggunakan `WHERE` saat melakukan `UPDATE`? Berikan contoh skenario nyata di aplikasi e-commerce jika lupa menggunakan `WHERE`!
2. Tuliskan query untuk menaikkan umur semua user yang tinggal di Bandung sebesar 1 tahun. Jelaskan mengapa query ini menggunakan ekspresi `umur = umur + 1` dan bukan nilai langsung!

5. DELETE — Menghapus Data

■ Penjelasan Mendalam

DELETE FROM digunakan untuk **menghapus satu atau lebih baris** dari tabel. `DELETE` tanpa `WHERE` adalah bencana — ia akan menghapus **semua data** di tabel. Data yang dihapus **tidak bisa dikembalikan** kecuali menggunakan `TRANSACTION (ROLLBACK)`.

■■ Perbedaan DELETE vs TRUNCATE vs DROP

Perintah	Fungsi	Bisa di-rollback?	Hapus Struktur?
DELETE	Hapus baris tertentu (dengan <code>WHERE</code>)	Ya (dalam <code>TRANSACTION</code>)	Tidak
TRUNCATE	Hapus semua baris, lebih cepat	Tergantung DB	Tidak
DROP	Hapus seluruh tabel beserta strukturnya	Tidak	Ya

Contoh 1 — Hapus berdasarkan ID

```
DELETE FROM users WHERE id = 6;
```

Contoh 2 — Hapus berdasarkan kondisi

```
DELETE FROM users WHERE umur < 21;
```

Contoh 3 — Hapus semua data (BERBAHAYA!)

```
DELETE FROM users; -- Semua baris terhapus, tabel masih ada tapi kosong
```

■ LATIHAN

Pilihan Ganda:

1. Apa perbedaan utama antara DELETE FROM users; dan DROP TABLE users;?
 - A. Keduanya identik
 - B. DELETE menghapus baris, DROP menghapus seluruh tabel termasuk strukturnya
 - C. DROP menghapus baris, DELETE menghapus strukturnya
 - D. DELETE hanya bisa menghapus satu baris
2. Query mana yang menghapus semua user yang berumur di atas 25?
 - A. REMOVE FROM users WHERE umur > 25;
 - B. DELETE users WHERE umur > 25;
 - C. DELETE FROM users WHERE umur > 25;
 - D. DELETE * FROM users WHERE umur > 25;
3. Apa yang terjadi jika DELETE FROM users; dijalankan tanpa WHERE?
 - A. Query error karena WHERE wajib ada
 - B. Hanya baris pertama yang terhapus
 - C. Semua baris terhapus, tabel menjadi kosong
 - D. Tabel ikut terhapus

Esai:

1. Jelaskan perbedaan antara DELETE, TRUNCATE, dan DROP TABLE dari sisi fungsi, kecepatan, dan kemampuan pemulihan data!
2. Bayangkan kamu bekerja di tim backend dan tidak sengaja menjalankan DELETE FROM orders; tanpa WHERE di database production. Apa langkah-langkah yang bisa dilakukan untuk mencegah dan memulihkan situasi seperti ini?

6. ORDER BY — Mengurutkan Data

■ Penjelasan Mendalam

ORDER BY digunakan untuk **mengurutkan hasil query** berdasarkan satu atau lebih kolom. Secara default, urutan adalah **ascending (ASC)** — dari kecil ke besar atau A ke Z. Gunakan **DESC** untuk membalik urutan. Jika tidak ada ORDER BY, urutan data tidak dijamin konsisten.

Contoh 1 — Ascending (kecil ke besar)

```
SELECT * FROM users ORDER BY umur ASC;
```

id	nama	umur	kota
1	Budi	20	Bandung
3	Rina	22	Bandung

2	Andi	25	Jakarta
4	Sari	30	Surabaya

Contoh 2 — Descending (besar ke kecil)

```
SELECT * FROM users ORDER BY umur DESC;
```

Contoh 3 — Multi-column ORDER BY

```
SELECT * FROM users ORDER BY kota ASC, umur DESC;
-- Urutkan berdasarkan kota (A-Z), jika kota sama maka umur dari terbesar
```

■ LATIHAN

Pilihan Ganda:

1. Apa urutan default jika ORDER BY tidak menggunakan ASC atau DESC?
 - A. Descending (besar ke kecil)
 - B. Ascending (kecil ke besar)
 - C. Acak
 - D. Berdasarkan urutan insert
2. Query mana yang menampilkan user dari yang paling tua ke yang paling muda?
 - A. SELECT * FROM users ORDER BY umur ASC;
 - B. SELECT * FROM users SORT BY umur DESC;
 - C. SELECT * FROM users ORDER BY umur DESC;
 - D. SELECT * FROM users ORDER umur DESC;
3. Apa hasil dari ORDER BY kota ASC, umur DESC?
 - A. Urutkan umur dari besar, lalu kota dari A
 - B. Urutkan kota dari A-Z, jika kota sama urutkan umur dari besar ke kecil
 - C. Urutkan berdasarkan kota saja, umur diabaikan
 - D. Error karena tidak bisa ORDER BY dua kolom

Esai:

1. Jelaskan kapan penggunaan ORDER BY dengan dua kolom berguna dalam konteks aplikasi nyata! Berikan satu contoh skenario dari aplikasi e-commerce!
2. Apakah ORDER BY mempengaruhi data yang tersimpan di database? Jelaskan apa yang sebenarnya terjadi ketika kita menggunakan ORDER BY!

7. LIMIT & OFFSET — Membatasi Jumlah Data

■ Penjelasan Mendalam

LIMIT membatasi jumlah baris yang dikembalikan. **OFFSET** digunakan bersama LIMIT untuk **melewati sejumlah baris** dari awal — menjadi dasar teknik **pagination** (halaman 1, 2, 3, dst).

■ Formula Pagination: $\text{OFFSET} = (\text{nomor_halaman} - 1) \times \text{jumlah_per_halaman}$ Contoh: halaman 3, 10 item per halaman → LIMIT 10 OFFSET 20

Contoh 1 — Ambil 2 baris pertama

```
SELECT * FROM users LIMIT 2;
```

id	nama	umur	kota
1	Budi	20	Bandung

2	Andi	25	Jakarta
---	------	----	---------

Contoh 2 — Mulai dari baris ke-3 (OFFSET 2)

```
SELECT * FROM users LIMIT 2 OFFSET 2;
```

id	nama	umur	kota
3	Rina	22	Bandung
4	Sari	30	Surabaya

Contoh 3 — Top-N: 2 user tertua

```
SELECT * FROM users ORDER BY umur DESC LIMIT 2;
```

id	nama	umur	kota
4	Sari	30	Surabaya
2	Andi	25	Jakarta

■ LATIHAN

Pilihan Ganda:

1. Apa kegunaan utama dari klausa LIMIT?
 - A. Membatasi jumlah kolom yang ditampilkan
 - B. Membatasi jumlah baris yang dikembalikan query
 - C. Membatasi ukuran file database
 - D. Membatasi jumlah query per detik
2. Jika kita ingin menampilkan halaman ke-2 dengan 2 data per halaman, query yang tepat adalah?
 - A. `SELECT * FROM users LIMIT 2 OFFSET 1;`
 - B. `SELECT * FROM users LIMIT 2 OFFSET 2;`
 - C. `SELECT * FROM users LIMIT 4;`
 - D. `SELECT * FROM users OFFSET 2;`
3. Query `SELECT * FROM users ORDER BY umur ASC LIMIT 1;` akan mengembalikan apa?
 - A. User dengan umur terbesar
 - B. User pertama yang dimasukkan
 - C. User dengan umur terkecil
 - D. Semua user diurutkan umur

Esai:

1. Jelaskan konsep pagination menggunakan LIMIT dan OFFSET! Tuliskan formula untuk menghitung OFFSET berdasarkan nomor halaman dan berikan contoh konkretnya!
2. Apa potensi masalah performa dari OFFSET yang sangat besar (misalnya OFFSET 1000000)? Jelaskan dan sebutkan alternatif yang lebih baik!

8. COUNT — Menghitung Jumlah Baris

■ Penjelasan Mendalam

COUNT adalah fungsi agregat yang **menghitung jumlah baris**. **COUNT(*)** menghitung semua baris termasuk NULL, sedangkan **COUNT(nama_kolom)** hanya menghitung baris di mana kolom tersebut tidak NULL.

■ Contoh Query

Hitung semua baris

```
SELECT COUNT(*) FROM users;  
-- Hasil: 4
```

Hitung dengan kondisi

```
SELECT COUNT(*) FROM users WHERE kota = 'Bandung';  
-- Hasil: 2
```

Beri nama alias

```
SELECT COUNT(*) AS total_user FROM users;  
-- Hasil: total_user = 4
```

■ LATIHAN

Pilihan Ganda:

1. Apa perbedaan antara COUNT(*) dan COUNT(kolom)?
 - A. Tidak ada perbedaan
 - B. COUNT(*) menghitung semua baris termasuk NULL, COUNT(kolom) mengabaikan NULL
 - C. COUNT(*) lebih lambat
 - D. COUNT(kolom) menghitung semua baris termasuk NULL
2. Berapa hasil dari SELECT COUNT(*) FROM users WHERE umur >= 22;?
 - A. 1
 - B. 2
 - C. 3
 - D. 4
3. Bagaimana cara memberi nama alias pada hasil COUNT?
 - A. COUNT(*) NAME total
 - B. COUNT(*) RENAME total
 - C. COUNT(*) AS total
 - D. COUNT(*) = total

Esai:

1. Kapan penggunaan COUNT menjadi penting dalam sebuah aplikasi? Berikan minimal 2 contoh use case nyata!
2. Jelaskan apa yang terjadi jika sebuah kolom berisi nilai NULL dan kita menggunakan COUNT(kolom) versus COUNT(*)!

9. GROUP BY — Mengelompokkan Data

■ Penjelasan Mendalam

GROUP BY digunakan untuk **mengelompokkan baris-baris** yang memiliki nilai sama pada kolom tertentu, kemudian menerapkan fungsi agregat pada setiap kelompok. Setelah **GROUP BY**, setiap kelompok direpresentasikan oleh **satu baris** dalam hasil.

■ Aturan penting: Kolom yang ada di **SELECT** harus berupa kolom yang di-**GROUP-BY** atau hasil fungsi agregat.

Contoh 1 — Hitung user per kota

```
SELECT kota, COUNT(*) AS jumlah_user
FROM users
GROUP BY kota;
```

kota	jumlah_user
Bandung	2
Jakarta	1
Surabaya	1

Contoh 2 — Multiple agregat sekaligus

```
SELECT kota, COUNT(*) AS jumlah, AVG(umur) AS rata_umur, MAX(umur) AS tertua
FROM users
GROUP BY kota;
```

kota	jumlah	rata_umur	tertua
Bandung	2	21.0	22
Jakarta	1	25.0	25
Surabaya	1	30.0	30

■ LATIHAN

Pilihan Ganda:

1. Apa fungsi utama dari **GROUP BY**?
 - A. Mengurutkan data berdasarkan kolom
 - B. Mengelompokkan baris dengan nilai sama dan memungkinkan fungsi agregat
 - C. Memfilter kelompok berdasarkan kondisi
 - D. Menggabungkan dua tabel
2. Query **SELECT nama, kota, COUNT(*) FROM users GROUP BY kota;** valid atau tidak?
 - A. Valid, akan menampilkan nama, kota, dan jumlah
 - B. Tidak valid, karena nama tidak ada di **GROUP BY** dan bukan fungsi agregat
 - C. Valid, karena **COUNT(*)** sudah cukup
 - D. Tidak valid, karena **GROUP BY** hanya menerima satu kolom
3. Apa hasil dari **SELECT COUNT(*), kota FROM users GROUP BY kota ORDER BY COUNT(*) DESC**?
 - A. Semua user diurutkan berdasarkan kota
 - B. Jumlah user per kota, diurutkan dari yang terbanyak
 - C. Error karena tidak bisa **ORDER BY COUNT**
 - D. Jumlah user per kota, diurutkan A-Z

Esai:

1. Jelaskan mengapa kolom yang ada di SELECT harus merupakan kolom GROUP BY atau fungsi agregat!
2. Tuliskan query untuk menghitung jumlah user dan rata-rata umur per kota, kemudian urutkan berdasarkan jumlah user terbanyak!

10. CREATE TABLE — Membuat Tabel

■ Penjelasan Mendalam

CREATE TABLE digunakan untuk **mendefinisikan struktur tabel baru**. Di sinilah kamu menentukan nama tabel, nama kolom, tipe data, dan constraint (aturan).

Tipe Data	Kegunaan	Contoh
INT / INTEGER	Angka bulat	id, umur, stok
VARCHAR(n)	Teks dengan batas karakter	nama, email
TEXT	Teks panjang tanpa batas	deskripsi, konten
BOOLEAN	True/False	is_active, is_verified
DECIMAL(p,s)	Angka desimal presisi	harga (12000.50)
TIMESTAMP	Tanggal + waktu	created_at
UUID	ID unik universal	id di Supabase

Contoh — Buat tabel lengkap dengan constraint

```
CREATE TABLE products (  
  id SERIAL PRIMARY KEY,  
  nama VARCHAR(200) NOT NULL,  
  harga DECIMAL(15,2) NOT NULL,  
  stok INT DEFAULT 0,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

■ LATIHAN

Pilihan Ganda:

1. Tipe data mana yang paling tepat untuk menyimpan harga produk seperti 12000.50?
 - A. INT
 - B. VARCHAR
 - C. DECIMAL(10,2)
 - D. BOOLEAN
2. Apa fungsi DEFAULT dalam definisi kolom?
 - A. Membuat kolom wajib diisi
 - B. Memberikan nilai otomatis jika kolom tidak diisi saat INSERT
 - C. Menghapus nilai yang ada
 - D. Membuat kolom menjadi PRIMARY KEY
3. Apa itu SERIAL di PostgreSQL?
 - A. Tipe data untuk menyimpan seri angka

- B. Tipe data integer yang otomatis bertambah (auto-increment)
- C. Cara membuat tabel secara serial
- D. Constraint untuk memastikan nilai unik

Esai:

1. Jelaskan perbedaan antara VARCHAR(100) dan TEXT dalam SQL! Kapan sebaiknya menggunakan masing-masing?
2. Rancang struktur tabel students yang menyimpan data mahasiswa dengan minimal 6 kolom yang relevan dan tuliskan CREATE TABLE-nya!

11. PRIMARY KEY — Identitas Unik Data

■ Penjelasan Mendalam

PRIMARY KEY adalah constraint yang menjamin setiap baris memiliki **identitas yang unik dan tidak null**. Sebuah tabel hanya boleh memiliki **satu** primary key, tapi bisa terdiri dari beberapa kolom (composite PK).

■ Contoh Query

Surrogate Key (recommended)

```
id SERIAL PRIMARY KEY
-- Atau UUID:
id UUID DEFAULT gen_random_uuid() PRIMARY KEY
```

Composite Primary Key

```
CREATE TABLE enrollments (
  student_id INT,
  course_id INT,
  enrolled_at TIMESTAMP,
  PRIMARY KEY (student_id, course_id) -- Gabungan dua kolom
);
```

■ LATIHAN

Pilihan Ganda:

1. Berapa jumlah PRIMARY KEY maksimal dalam satu tabel?
 - A. Tidak terbatas
 - B. Maksimal 2
 - C. Tepat 1 (bisa composite dari beberapa kolom)
 - D. Maksimal 5
2. Apa yang membedakan PRIMARY KEY dengan UNIQUE constraint?
 - A. Tidak ada perbedaan
 - B. PRIMARY KEY tidak boleh NULL, UNIQUE boleh NULL (di beberapa database)
 - C. UNIQUE tidak boleh duplikat, PRIMARY KEY boleh duplikat
 - D. PRIMARY KEY hanya untuk angka
3. Mengapa UUID lebih sering digunakan sebagai Primary Key di aplikasi modern?
 - A. UUID lebih kecil ukurannya
 - B. UUID lebih mudah diingat
 - C. UUID unik secara global sehingga aman untuk sistem terdistribusi

D. UUID lebih cepat diproses

Esai:

1. Jelaskan perbedaan antara Natural Key dan Surrogate Key sebagai Primary Key!
2. Apa itu Composite Primary Key? Berikan contoh tabel nyata yang membutuhkannya!

12. DISTINCT — Nilai Unik

■ Penjelasan Mendalam

DISTINCT digunakan untuk **menghilangkan duplikat** dari hasil query, sehingga setiap nilai hanya muncul sekali. **DISTINCT** bekerja pada seluruh kombinasi kolom yang disebutkan.

■ Contoh Query

Kota unik

```
SELECT DISTINCT kota FROM users;  
  
-- Bandung, Jakarta, Surabaya (tanpa duplikat)
```

COUNT DISTINCT

```
SELECT COUNT(DISTINCT kota) AS jumlah_kota FROM users;  
  
-- Hasil: 3
```

■ LATIHAN

Pilihan Ganda:

1. Apa fungsi DISTINCT dalam query SELECT?
 - A. Mengurutkan nilai dari kecil ke besar
 - B. Menghilangkan baris duplikat dari hasil query
 - C. Memfilter berdasarkan kondisi tertentu
 - D. Menggabungkan nilai yang sama
2. Berapa hasil dari `SELECT COUNT(DISTINCT kota) FROM users;`?
 - A. 1
 - B. 2
 - C. 3
 - D. 4
3. Jika `SELECT DISTINCT kota, umur FROM users;` dijalankan, apa yang dianggap duplikat?
 - A. Baris dengan kota yang sama
 - B. Baris dengan umur yang sama
 - C. Baris dengan kombinasi kota DAN umur yang sama
 - D. Semua baris dianggap duplikat

Esai:

1. Apa perbedaan antara `SELECT DISTINCT kota` dan `SELECT kota FROM users GROUP BY kota`?
2. Berikan contoh use case nyata di aplikasi e-commerce di mana **DISTINCT** sangat berguna!

13. LIKE — Pencarian Pola Teks

■ Penjelasan Mendalam

LIKE digunakan untuk mencari data berdasarkan **pola teks** (pattern matching). Wildcard: **%** mewakili nol atau lebih karakter, **_** mewakili tepat satu karakter. PostgreSQL juga menyediakan **ILIKE** untuk pencarian tidak case-sensitive.

■ Contoh Query

Nama diawali huruf B

```
SELECT * FROM users WHERE nama LIKE 'B%';  
  
-- Hasil: Budi
```

Nama diakhiri huruf i

```
SELECT * FROM users WHERE nama LIKE '%i';  
  
-- Hasil: Budi, Andi
```

Kota mengandung 'ung'

```
SELECT * FROM users WHERE kota LIKE '%ung';  
  
-- Hasil: Bandung
```

■ LATIHAN

Pilihan Ganda:

1. Apa arti wildcard % dalam LIKE?
 - A. Tepat satu karakter apapun
 - B. Nol atau lebih karakter apapun
 - C. Hanya huruf kapital
 - D. Hanya angka
2. Query mana yang mencari user dengan nama yang mengandung huruf 'n'?
 - A. WHERE nama LIKE 'n';
 - B. WHERE nama LIKE 'n%';
 - C. WHERE nama LIKE '%n%';
 - D. WHERE nama LIKE '%n';
3. Apa perbedaan antara LIKE dan ILIKE di PostgreSQL?
 - A. Tidak ada perbedaan
 - B. ILIKE mengabaikan perbedaan huruf besar/kecil, LIKE tidak
 - C. LIKE lebih cepat dari ILIKE
 - D. ILIKE hanya untuk angka

Esai:

1. Jelaskan perbedaan antara wildcard % dan _ dalam LIKE, dan berikan contoh query menggunakan masing-masing!
2. Mengapa penggunaan LIKE '%kata%' (dengan % di depan) dapat menyebabkan masalah performa pada tabel besar?

14. IN — Filter dengan Banyak Nilai

■ Penjelasan Mendalam

IN digunakan untuk **memfilter baris** di mana nilai kolom cocok dengan **salah satu nilai** dalam sebuah daftar. Lebih bersih dan efisien dibandingkan menulis banyak kondisi OR. **NOT IN** adalah kebalikannya.

■ Contoh Query

Filter beberapa kota

```
SELECT * FROM users WHERE kota IN ('Bandung', 'Jakarta');
```

Filter beberapa ID

```
SELECT * FROM users WHERE id IN (1, 3, 4);
```

NOT IN

```
SELECT * FROM users WHERE kota NOT IN ('Bandung', 'Jakarta');
```

■ LATIHAN

Pilihan Ganda:

1. WHERE kota IN ('Bandung', 'Jakarta') setara dengan?
 - A. WHERE kota = 'Bandung' AND kota = 'Jakarta'
 - B. WHERE kota = 'Bandung' OR kota = 'Jakarta'
 - C. WHERE kota BETWEEN 'Bandung' AND 'Jakarta'
 - D. WHERE kota LIKE 'Bandung' OR 'Jakarta'
2. Berapa baris hasil dari SELECT * FROM users WHERE id IN (2, 4);?
 - A. 1
 - B. 2
 - C. 3
 - D. 4
3. Apa fungsi NOT IN?
 - A. Mengambil baris dengan nilai yang ada dalam daftar
 - B. Mengambil baris dengan nilai yang tidak ada dalam daftar
 - C. Membalik urutan daftar
 - D. Error karena NOT IN tidak ada di SQL

Esai:

1. Jelaskan mengapa IN lebih disarankan dibandingkan banyak kondisi OR dalam hal keterbacaan!
2. Apa risiko menggunakan NOT IN jika salah satu nilai dalam daftar adalah NULL?

15. BETWEEN — Filter Rentang Nilai

■ Penjelasan Mendalam

BETWEEN digunakan untuk memfilter baris di mana nilai kolom berada dalam **rentang tertentu, inklusif** (termasuk nilai batas bawah dan atas). Bisa digunakan untuk angka, tanggal, maupun teks.

■ Contoh Query

Rentang angka

```
SELECT * FROM users WHERE umur BETWEEN 21 AND 30;  
-- Rina (22), Andi (25), Sari (30) masuk – Budi (20) tidak
```

Rentang tanggal

```
SELECT * FROM orders WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31';
```

NOT BETWEEN


```
SELECT * FROM users WHERE umur NOT BETWEEN 21 AND 25;  
-- Budi (20) dan Sari (30)
```

■ LATIHAN

Pilihan Ganda:

1. WHERE umur BETWEEN 20 AND 25 apakah menyertakan umur 20 dan 25?
 - A. Tidak, keduanya tidak termasuk
 - B. Ya, keduanya termasuk (inklusif)
 - C. Hanya 20 yang termasuk
 - D. Hanya 25 yang termasuk
2. WHERE umur BETWEEN 21 AND 30 pada dataset standar menghasilkan berapa baris?
 - A. 1
 - B. 2
 - C. 3
 - D. 4
3. Ekspresi mana yang SETARA dengan WHERE umur BETWEEN 20 AND 25?
 - A. WHERE umur > 20 AND umur < 25
 - B. WHERE umur >= 20 AND umur <= 25
 - C. WHERE umur >= 20 OR umur <= 25
 - D. WHERE umur > 20 OR umur < 25

Esai:

1. Jelaskan penggunaan BETWEEN untuk kolom bertipe tanggal! Berikan contoh skenario laporan penjualan bulanan!
2. Apa yang dimaksud bahwa BETWEEN bersifat inklusif?

16. ALIAS (AS) — Mengganti Nama Sementara

■ Penjelasan Mendalam

AS digunakan untuk memberikan **nama sementara (alias)** pada kolom atau tabel dalam hasil query. Alias hanya berlaku dalam konteks query tersebut dan tidak mengubah nama asli di database.

■ Contoh Query

Alias pada kolom

```
SELECT nama AS username, umur AS usia FROM users;
```

Alias pada fungsi

```
SELECT kota, COUNT(*) AS total_pengguna, AVG(umur) AS rata_umur  
FROM users  
GROUP BY kota;
```

Alias pada tabel (saat JOIN)

```
SELECT u.nama, o.produk  
FROM users AS u  
JOIN orders AS o ON u.id = o.user_id;
```

■ LATIHAN

Pilihan Ganda:

1. Apakah alias mengubah nama kolom di database secara permanen?
 - A. Ya, nama kolom berubah permanen
 - B. Tidak, alias hanya berlaku pada hasil query tersebut
 - C. Tergantung jenis database
 - D. Hanya berubah jika menggunakan ALTER TABLE
2. Kata kunci AS dalam SQL bersifat?
 - A. Wajib ada, tidak bisa dihilangkan
 - B. Opsional, alias bisa ditulis langsung tanpa AS
 - C. Hanya untuk kolom, tidak bisa untuk tabel
 - D. Hanya bisa digunakan sekali per query
3. Dalam konteks JOIN, mengapa alias tabel sangat berguna?
 - A. Mempercepat eksekusi query
 - B. Menyingkat penulisan nama tabel yang panjang sehingga query lebih ringkas
 - C. Membuat tabel baru di database
 - D. Menghindari duplikasi data

Esai:

1. Jelaskan pentingnya penggunaan alias dalam query yang kompleks! Berikan contoh dengan dan tanpa alias!
2. Bolehkah menggunakan alias kolom di klausa WHERE? Mengapa? Jelaskan dengan menyebut urutan eksekusi SQL!

17. SUM & AVG — Operasi Matematika

■ Penjelasan Mendalam

SUM menghitung total penjumlahan semua nilai, sedangkan **AVG** menghitung nilai rata-rata. Keduanya fungsi agregat yang mengabaikan nilai NULL. Selain itu ada **MIN** dan **MAX**.

■ Contoh Query

Total umur

```
SELECT SUM(umur) AS total_umur FROM users;
-- Hasil: 97 (20+25+22+30)
```

Rata-rata umur

```
SELECT AVG(umur) AS rata_rata_umur FROM users;
-- Hasil: 24.25
```

Semua agregat sekaligus

```
SELECT MIN(umur) AS termuda, MAX(umur) AS tertua,
SUM(umur) AS total, AVG(umur) AS rata, COUNT(*) AS jumlah
FROM users;
```

■ LATIHAN

Pilihan Ganda:

1. Apa hasil dari `SELECT SUM(umur) FROM users`?
 - A. 24.25
 - B. 97
 - C. 4
 - D. 30
2. Bagaimana fungsi agregat memperlakukan nilai NULL?
 - A. NULL dianggap 0
 - B. NULL menyebabkan seluruh hasil menjadi NULL
 - C. NULL diabaikan dalam perhitungan
 - D. NULL dianggap 1
3. Query mana yang menghasilkan umur tertua dari tabel `users`?
 - A. `SELECT TOP(umur) FROM users;`
 - B. `SELECT MAX(umur) FROM users;`
 - C. `SELECT umur ORDER BY umur DESC LIMIT 1;`
 - D. `SELECT HIGHEST(umur) FROM users;`

Esai:

1. Jelaskan perbedaan antara `AVG(kolom)` dan `SUM(kolom)/COUNT(*)`. Kapan hasilnya bisa berbeda?
2. Tuliskan query untuk mendapatkan total pendapatan, rata-rata transaksi, transaksi terbesar, dan terkecil dari tabel `orders`!

18. HAVING — Filter Setelah GROUP BY

■ Penjelasan Mendalam

HAVING digunakan untuk **memfilter hasil GROUP BY** berdasarkan kondisi pada fungsi agregat. Perbedaan mendasar: **WHERE** memfilter baris *sebelum* pengelompokan, sedangkan **HAVING** memfilter *kelompok* yang sudah terbentuk.

■ Analogi: **WHERE** menyaring bahan mentah sebelum dimasak, **HAVING** menyaring hidangan yang sudah jadi.

Contoh 1 — Kota dengan lebih dari 1 user

```
SELECT kota, COUNT(*) AS jumlah
FROM users
GROUP BY kota
HAVING COUNT(*) > 1;
```

kota	jumlah
Bandung	2

Jakarta dan Surabaya tidak muncul karena masing-masing hanya punya 1 user.

Contoh 2 — Kota dengan rata-rata umur di atas 22

```
SELECT kota, AVG(umur) AS rata_umur
FROM users
GROUP BY kota
HAVING AVG(umur) > 22;
```

kota	rata_umur
Jakarta	25.0

■ LATIHAN

Pilihan Ganda:

1. Apa perbedaan utama antara WHERE dan HAVING?
 - A. WHERE untuk teks, HAVING untuk angka
 - B. WHERE memfilter baris sebelum GROUP BY, HAVING memfilter kelompok setelah GROUP BY
 - C. HAVING lebih cepat dari WHERE
 - D. WHERE dan HAVING identik
2. Apakah kita bisa menggunakan HAVING tanpa GROUP BY?
 - A. Tidak bisa sama sekali
 - B. Bisa, tapi jarang berguna karena seluruh tabel dianggap satu kelompok
 - C. Hanya bisa di PostgreSQL
 - D. Bisa dan sangat sering digunakan
3. Manakah urutan klausa SQL yang benar?
 - A. SELECT → HAVING → WHERE → GROUP BY → FROM
 - B. FROM → WHERE → GROUP BY → HAVING → SELECT
 - C. SELECT → WHERE → FROM → GROUP BY → HAVING
 - D. FROM → HAVING → WHERE → SELECT → GROUP BY

Esai:

1. Jelaskan dengan analogi sehari-hari perbedaan antara WHERE dan HAVING! Kemudian berikan satu contoh query yang menggunakan keduanya sekaligus!
2. Tuliskan query untuk menemukan kota yang memiliki rata-rata umur user di atas 20, urutkan dari rata-rata umur tertinggi!

19. JOIN — Menggabungkan Tabel

■ Penjelasan Mendalam

JOIN adalah operasi untuk **menggabungkan data dari dua atau lebih tabel** berdasarkan kolom yang berelasi. Ini adalah salah satu fitur terpenting dalam relational database.

Jenis JOIN	Penjelasan
INNER JOIN	Hanya baris yang cocok di KEDUA tabel
LEFT JOIN	Semua baris dari tabel kiri + baris cocok dari kanan (NULL jika tidak ada)
RIGHT JOIN	Semua baris dari tabel kanan + baris cocok dari kiri
FULL OUTER JOIN	Semua baris dari kedua tabel

Contoh 1 — INNER JOIN

```
SELECT users.nama, orders.produk, orders.harga
FROM users
INNER JOIN orders ON users.id = orders.user_id;
```

nama	produk	harga
Budi	Laptop	12.000.000

Andi	Mouse	150.000
Budi	Keyboard	450.000
Rina	Monitor	2.500.000

Sari tidak muncul karena tidak ada pesanan dengan `user_id = 4`.

Contoh 2 — LEFT JOIN (semua user termasuk yang tidak punya pesanan)

```
SELECT users.nama, orders.produk
FROM users
LEFT JOIN orders ON users.id = orders.user_id;
```

nama	produk
Budi	Laptop
Budi	Keyboard
Andi	Mouse
Rina	Monitor
Sari	NULL

Sari muncul dengan nilai NULL karena tidak ada pesanan. Ini yang membedakan LEFT JOIN dari INNER JOIN.

■ LATIHAN

Pilihan Ganda:

- Apa perbedaan antara INNER JOIN dan LEFT JOIN?
 - Tidak ada perbedaan
 - INNER JOIN hanya mengembalikan baris yang cocok di kedua tabel; LEFT JOIN mengembalikan semua baris dari tabel kiri meskipun tidak ada pasangannya
 - LEFT JOIN lebih cepat dari INNER JOIN
 - INNER JOIN mengambil semua baris dari tabel kiri
- Mengapa Sari muncul dengan nilai NULL pada contoh LEFT JOIN?
 - Data produk Sari belum dimasukkan ke tabel orders
 - LEFT JOIN memaksa semua baris dari tabel kiri muncul, meski tidak ada data di tabel kanan
 - Sari memiliki produk tapi harganya 0
 - JOIN selalu menghasilkan NULL untuk baris terakhir
- Kondisi `ON users.id = orders.user_id` berfungsi untuk?
 - Menyaring baris berdasarkan umur
 - Menentukan kolom mana yang dijadikan kunci penghubung antara dua tabel
 - Membuat kolom baru di tabel
 - Mengurutkan hasil

Esai:

- Jelaskan kapan sebaiknya menggunakan LEFT JOIN dibandingkan INNER JOIN! Berikan contoh skenario nyata!
- Mengapa data harus dipisah ke banyak tabel padahal nanti harus di-JOIN kembali? Jelaskan keuntungannya!

20. SUBQUERY — Query di Dalam Query

■ Penjelasan Mendalam

Subquery (inner query / nested query) adalah **query SQL yang berada di dalam query lain**. Subquery dieksekusi terlebih dahulu, dan hasilnya digunakan oleh query luar. Bisa berada di klausa WHERE, FROM, atau SELECT.

■ Contoh Query

Lebih tua dari rata-rata

```
SELECT * FROM users
WHERE umur > (SELECT AVG(umur) FROM users);
-- Subquery menghasilkan 24.25, query luar filter > 24.25
```

User dengan nilai maximum

```
SELECT * FROM users
WHERE umur = (SELECT MAX(umur) FROM users);
-- Hasil: Sari (umur 30)
```

Subquery dengan IN

```
SELECT * FROM users
WHERE id IN (SELECT user_id FROM orders WHERE harga > 1000000);
-- Budi dan Rina yang beli di atas 1 juta
```

■ LATIHAN

Pilihan Ganda:

1. Bagian mana yang dieksekusi pertama dalam query dengan subquery?
 - A. Query luar (outer query)
 - B. Subquery (inner query)
 - C. Keduanya paralel
 - D. Tergantung posisi subquery
2. Subquery mana yang menghasilkan user dengan umur tertua?
 - A. WHERE umur = (SELECT MIN(umur) FROM users)
 - B. WHERE umur > (SELECT AVG(umur) FROM users)
 - C. WHERE umur = (SELECT MAX(umur) FROM users)
 - D. WHERE umur IN (SELECT umur FROM users)
3. Kapan sebaiknya menggunakan JOIN dibandingkan Subquery?
 - A. Selalu gunakan Subquery
 - B. JOIN umumnya lebih efisien untuk menggabungkan data antar tabel; Subquery lebih cocok untuk kondisi berbasis agregat
 - C. Keduanya selalu identik
 - D. Subquery selalu lebih cepat

Esai:

1. Tuliskan query untuk menemukan user yang usianya di atas rata-rata. Bandingkan dengan pendekatan lain!
2. Apa yang dimaksud dengan correlated subquery? Jelaskan perbedaannya dengan subquery biasa!

21. INDEX — Mempercepat Query

■ Penjelasan Mendalam

Index adalah struktur data tambahan yang dibuat pada kolom tertentu untuk **mempercepat pencarian data**. Analogi: indeks di belakang buku. Tanpa index, database melakukan **Full Table Scan** — membaca setiap baris satu per satu.

■ Contoh Query

Buat index pada kolom nama

```
CREATE INDEX idx_users_nama ON users(nama);  
-- Setelah ini WHERE nama = 'Budi' jauh lebih cepat
```

Composite index

```
CREATE INDEX idx_users_kota_umur ON users(kota, umur);  
-- Mempercepat query yang filter kota DAN umur
```

Hapus index

```
DROP INDEX idx_users_nama;
```

■ LATIHAN

Pilihan Ganda:

1. Apa analogi yang paling tepat untuk INDEX dalam database?
 - A. Sampul buku
 - B. Indeks di belakang buku yang membantu menemukan kata di halaman tertentu
 - C. Daftar isi buku
 - D. Bookmark halaman favorit
2. Kapan INDEX justru dapat memperlambat database?
 - A. Saat menjalankan SELECT dengan WHERE
 - B. Saat melakukan operasi INSERT, UPDATE, atau DELETE
 - C. Saat menggunakan ORDER BY
 - D. Saat melakukan JOIN
3. Mengapa tidak disarankan membuat index pada kolom boolean is_active?
 - A. Boolean tidak bisa di-index
 - B. Index tidak efisien pada kolom dengan sedikit variasi nilai
 - C. Boolean memperlambat pembuatan index
 - D. Kolom boolean tidak bisa dipakai di WHERE

Esai:

1. Jelaskan konsep Full Table Scan dan bagaimana index mencegah hal tersebut!
2. Kapan kamu TIDAK sebaiknya membuat index? Sebutkan minimal 3 kondisi!

22. UNION & UNION ALL — Menggabungkan Hasil Query

■ Penjelasan Mendalam

UNION menggabungkan hasil dari dua atau lebih query menjadi satu hasil dan **menghapus duplikat**. **UNION ALL** melakukan hal sama tapi **mempertahankan duplikat** dan lebih cepat. Syarat: kedua query harus memiliki jumlah kolom yang sama dan tipe data yang kompatibel.

■ Contoh Query

UNION (hapus duplikat)

```
SELECT nama FROM users WHERE kota = 'Bandung'  
  
UNION  
  
SELECT nama FROM users WHERE umur > 25;
```

UNION dari dua tabel berbeda

```
SELECT nama AS entitas, 'user' AS jenis FROM users  
  
UNION  
  
SELECT produk AS entitas, 'produk' AS jenis FROM orders;
```

■ LATIHAN

Pilihan Ganda:

1. Apa perbedaan utama antara UNION dan UNION ALL?
 - A. UNION ALL lebih lambat dari UNION
 - B. UNION menghapus duplikat, UNION ALL mempertahankan duplikat
 - C. UNION bisa menggabungkan tabel berbeda, UNION ALL tidak
 - D. Tidak ada perbedaan
2. Syarat apa yang harus dipenuhi agar dua query bisa digabungkan dengan UNION?
 - A. Kedua tabel harus memiliki PRIMARY KEY yang sama
 - B. Kedua query harus menghasilkan jumlah kolom yang sama dan tipe data kompatibel
 - C. Kedua query harus FROM tabel yang sama
 - D. Keduanya harus memiliki kondisi WHERE yang sama
3. Kapan sebaiknya menggunakan UNION ALL dibanding UNION?
 - A. Ketika kita ingin duplikat dihapus
 - B. Ketika kita tahu tidak ada duplikat atau ingin mempertahankannya, karena UNION ALL lebih cepat
 - C. UNION ALL selalu lebih buruk
 - D. Ketika menggabungkan lebih dari 3 query

Esai:

1. Jelaskan mengapa UNION ALL umumnya lebih cepat daripada UNION!
2. Berikan skenario nyata di mana kamu perlu menggunakan UNION untuk menggabungkan data dari dua tabel berbeda!

23. CASE — Logika Kondisional (IF-ELSE)

■ Penjelasan Mendalam

CASE adalah ekspresi SQL yang memungkinkan **logika kondisional** (seperti IF-ELSE) langsung di dalam query. Sangat berguna untuk mengkategorikan, mengklasifikasikan, atau mentransformasi data secara dinamis.

■ Contoh Query

Kategorisasi umur

```
SELECT nama, umur,  
CASE  
WHEN umur < 21 THEN 'Remaja'  
WHEN umur BETWEEN 21 AND 25 THEN 'Dewasa Muda'  
ELSE 'Dewasa'  
END AS kategori_umur  
FROM users;
```

CASE dalam agregat (Pivot)

```
SELECT  
COUNT(CASE WHEN kota = 'Bandung' THEN 1 END) AS jumlah_bandung,  
COUNT(CASE WHEN kota = 'Jakarta' THEN 1 END) AS jumlah_jakarta,  
COUNT(CASE WHEN kota = 'Surabaya' THEN 1 END) AS jumlah_surabaya  
FROM users;
```

■ LATIHAN

Pilihan Ganda:

1. Apa padanan CASE dalam bahasa pemrograman umum?
 - A. For loop
 - B. While loop
 - C. IF-ELSE atau switch statement
 - D. Try-catch
2. Apa yang dikembalikan jika tidak ada kondisi WHEN yang terpenuhi dan tidak ada ELSE?
 - A. Error
 - B. 0
 - C. NULL
 - D. String kosong
3. Teknik mengubah nilai baris menjadi kolom menggunakan CASE disebut?
 - A. Normalisasi
 - B. Pivot
 - C. Partisi
 - D. Transpose JOIN

Esai:

1. Tuliskan query untuk mengkategorikan harga pesanan di tabel orders menjadi Budget, Mid-range, dan Premium!
2. Jelaskan penggunaan CASE di dalam fungsi agregat! Berikan contoh query yang menghitung jumlah user berdasarkan kategori umur!

24. VIEW — Tabel Virtual

■ Penjelasan Mendalam

VIEW adalah **query yang disimpan** sebagai objek dalam database dan bisa dipanggil seperti tabel biasa. View tidak menyimpan data secara fisik — setiap kali view diakses, query di baliknya dieksekusi ulang.

■ Contoh Query

Buat view user Bandung

```
CREATE VIEW user_bandung AS
SELECT id, nama, umur
FROM users
WHERE kota = 'Bandung';

-- Penggunaan:
SELECT * FROM user_bandung;
```

View dengan JOIN

```
CREATE VIEW user_orders AS
SELECT u.nama, u.kota, o.produk, o.harga
FROM users u
JOIN orders o ON u.id = o.user_id;

-- Penggunaan cukup:
SELECT * FROM user_orders;
```

Hapus dan update view

```
DROP VIEW user_bandung;

CREATE OR REPLACE VIEW user_bandung AS
SELECT * FROM users WHERE kota = 'Bandung';
```

■ LATIHAN

Pilihan Ganda:

1. Apakah VIEW menyimpan data secara fisik di database?
 - A. Ya, data disalin ke storage terpisah
 - B. Tidak, view hanya menyimpan definisi query
 - C. Ya, tapi hanya data terbaru
 - D. Tergantung jenis database
2. Apa keuntungan utama menggunakan VIEW untuk keamanan?
 - A. View mengenkripsi data secara otomatis
 - B. View memungkinkan pembatasan akses hanya pada kolom/baris tertentu tanpa mengekspos seluruh tabel
 - C. View membuat data tidak bisa dihapus
 - D. View otomatis membackup data
3. Perintah apa yang digunakan untuk menghapus sebuah VIEW?
 - A. DELETE VIEW nama_view;
 - B. REMOVE VIEW nama_view;
 - C. DROP VIEW nama_view;
 - D. TRUNCATE VIEW nama_view;

Esai:

1. Jelaskan perbedaan antara View biasa dan Materialized View! Kapan sebaiknya menggunakan Materialized View?
2. Bagaimana VIEW dapat meningkatkan keamanan database dalam konteks aplikasi multi-user?

25. ALTER TABLE — Mengubah Struktur Tabel

■ Penjelasan Mendalam

ALTER TABLE digunakan untuk **memodifikasi struktur tabel** yang sudah ada tanpa harus menghapus dan membuat ulang tabel. Beberapa operasi bersifat destruktif dan tidak bisa di-undo.

■ Contoh Query

Operasi ALTER TABLE umum

```
ALTER TABLE users ADD COLUMN email VARCHAR(200) UNIQUE;
ALTER TABLE users DROP COLUMN email;
ALTER TABLE users RENAME COLUMN nama TO full_name;
ALTER TABLE users ALTER COLUMN umur TYPE BIGINT;
```

Tambah kolom dengan DEFAULT

```
ALTER TABLE users ADD COLUMN is_active BOOLEAN DEFAULT TRUE;
-- Semua baris yang ada otomatis mendapat nilai TRUE
```

■ LATIHAN

Pilihan Ganda:

1. Apa yang terjadi pada data yang sudah ada ketika menambahkan kolom baru?
 - A. Semua baris terhapus otomatis
 - B. Baris yang ada mendapat nilai NULL (atau DEFAULT jika didefinisikan)
 - C. Kita harus mengisi nilai kolom baru secara manual
 - D. Error karena tidak bisa menambah kolom pada tabel berisi data
2. Query mana yang benar untuk mengganti nama kolom nama menjadi full_name di PostgreSQL?
 - A. ALTER TABLE users CHANGE nama full_name VARCHAR(100);
 - B. ALTER TABLE users RENAME nama TO full_name;
 - C. ALTER TABLE users RENAME COLUMN nama TO full_name;
 - D. UPDATE TABLE users SET column_name = 'full_name' WHERE column = 'nama';
3. Apa risiko menjalankan ALTER TABLE users DROP COLUMN email?
 - A. Tidak ada risiko, kolom bisa dikembalikan
 - B. Kolom email dan semua datanya terhapus permanen (kecuali dalam TRANSACTION)
 - C. Hanya definisi kolom yang terhapus, datanya aman
 - D. Error, tidak bisa menghapus kolom yang ada datanya

Esai:

1. Jelaskan strategi yang aman untuk melakukan perubahan kolom pada tabel production yang berisi jutaan baris!
2. Apa perbedaan perintah ALTER TABLE di MySQL dengan PostgreSQL dalam hal sintaks penggantian nama kolom?

26. NOT NULL & DEFAULT — Constraint Validasi

■ Penjelasan Mendalam

NOT NULL memastikan sebuah kolom **tidak boleh dikosongkan**. **DEFAULT** adalah nilai yang otomatis diisikan jika kolom tidak disebutkan saat INSERT. Penting: **NULL** ≠ **String kosong** " — NULL berarti tidak ada nilai sama sekali.

■ Contoh Query

Definisi saat CREATE TABLE

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  nama VARCHAR(100) NOT NULL,  
  email VARCHAR(200) NOT NULL UNIQUE,  
  umur INT DEFAULT 18,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

IS NULL dan IS NOT NULL

```
-- Mencari user yang belum mengisi umur  
SELECT * FROM users WHERE umur IS NULL;  
  
-- SALAH: WHERE umur = NULL (tidak akan menemukan apapun!)
```

■ LATIHAN

Pilihan Ganda:

1. Apa perbedaan antara nilai NULL dan string kosong dalam SQL?
 - A. Keduanya identik
 - B. NULL berarti tidak ada nilai sama sekali, string kosong adalah nilai yang ada tapi isinya kosong
 - C. String kosong lebih baik dari NULL
 - D. NULL sama dengan angka 0
2. Bagaimana cara memeriksa apakah sebuah kolom bernilai NULL?
 - A. WHERE kolom = NULL
 - B. WHERE kolom == NULL
 - C. WHERE kolom IS NULL
 - D. WHERE ISNULL(kolom)
3. Jika kolom umur memiliki DEFAULT 18 dan kita INSERT tanpa menyebutkan umur, apa yang terjadi?
 - A. Error karena umur tidak diisi
 - B. Umur otomatis diisi dengan nilai 18
 - C. Umur bernilai NULL
 - D. Umur diisi 0

Esai:

1. Jelaskan mengapa menggunakan NULL bisa menjadi sumber bug yang sulit ditemukan dalam aplikasi!
2. Rancang struktur kolom yang tepat untuk tabel payments (pembayaran) dengan mempertimbangkan NOT NULL dan DEFAULT!

27. TRANSACTION — Keamanan Perubahan Data

■ Penjelasan Mendalam

TRANSACTION mengelompokkan beberapa operasi SQL menjadi satu unit kerja yang bersifat **atomik** — semua operasi berhasil, atau semua dibatalkan. Konsep ACID: **A**tomicity, **C**onsistency, **I**solation, **D**urability.

■ Contoh Query

ROLLBACK — batalkan perubahan

```
BEGIN;
UPDATE users SET umur = 999 WHERE id = 1;
-- Ups, salah!
ROLLBACK;
-- Data kembali seperti semula
```

Transfer uang (use case klasik)

```
BEGIN;
-- Kurangi saldo pengirim
UPDATE accounts SET saldo = saldo - 100000 WHERE user_id = 1;
-- Tambah saldo penerima
UPDATE accounts SET saldo = saldo + 100000 WHERE user_id = 2;
COMMIT; -- Jika sukses
-- ROLLBACK; -- Jika ada error di tengah jalan
```

■ LATIHAN

Pilihan Ganda:

1. Apa yang dimaksud dengan sifat Atomicity dalam ACID?
 - A. Transaksi berjalan sangat cepat
 - B. Semua operasi dalam transaksi harus berhasil semua, atau dibatalkan semua — tidak ada yang setengah jadi
 - C. Transaksi tidak bisa dibagi menjadi sub-transaksi
 - D. Hanya satu transaksi yang bisa berjalan
2. Apa yang terjadi pada data jika kita menjalankan ROLLBACK?
 - A. Data terhapus permanen
 - B. Semua perubahan sejak BEGIN dibatalkan dan data kembali ke kondisi sebelum transaksi
 - C. Hanya operasi terakhir yang dibatalkan
 - D. Database reboot
3. Mengapa TRANSACTION sangat penting untuk operasi transfer uang?
 - A. Membuat transfer lebih cepat
 - B. Memastikan tidak ada uang yang hilang jika terjadi error di tengah proses
 - C. Mengenkripsi data transfer
 - D. Mencatat log transfer secara otomatis

Esai:

1. Jelaskan seluruh prinsip ACID dalam TRANSACTION! Mengapa setiap prinsip penting untuk integritas data?

2. Berikan skenario nyata selain transfer uang di mana TRANSACTION sangat krusial! Tuliskan query lengkap!

28. NORMALISASI DATABASE — Merancang Struktur yang Efisien

■ Penjelasan Mendalam

Normalisasi adalah proses mendesain struktur tabel agar data tersimpan secara efisien, tidak redundan, dan konsisten. Ada tiga level utama: **1NF**, **2NF**, dan **3NF**.

■ Contoh Query

1NF — Nilai harus atomic (satu sel = satu nilai)

```
-- ■ MELANGGAR 1NF:
-- hobi = 'Membaca, Olahraga, Coding' (banyak nilai dalam satu sel)

-- ■ SOLUSI: pisah ke tabel terpisah
CREATE TABLE user_hobi (
  user_id INT REFERENCES users(id),
  hobi VARCHAR(100)
);
```

3NF — Hilangkan ketergantungan transitif

```
-- ■ MELANGGAR 3NF: nama_kota & kode_pos tergantung kota_id, bukan user.id
-- Tabel users: (id, nama, kota_id, nama_kota, kode_pos)

-- ■ SOLUSI: pisah tabel kota
CREATE TABLE kota (
  id SERIAL PRIMARY KEY,
  nama_kota VARCHAR(100),
  kode_pos VARCHAR(10)
);

-- users hanya simpan kota_id:
CREATE TABLE users (id SERIAL PK, nama VARCHAR(100), kota_id INT REFERENCES kota(id));
```

■ LATIHAN

Pilihan Ganda:

- Sebuah tabel memiliki kolom hobi yang berisi nilai 'Membaca, Olahraga, Coding' dalam satu sel. Aturan Normal Form mana yang dilanggar?
 - 2NF
 - 3NF
 - 1NF
 - 4NF
- Apa yang dimaksud dengan ketergantungan transitif yang harus dihilangkan pada 3NF?
 - Kolom A bergantung pada Primary Key
 - Kolom C bergantung pada Kolom B, dan Kolom B bergantung pada Primary Key — sehingga C bergantung tidak langsung pada PK
 - Dua tabel saling bergantung

D. Semua kolom harus bergantung pada foreign key

3. Apa keuntungan memisahkan data kota ke tabel tersendiri?

- A. Query menjadi lebih sederhana tanpa JOIN
- B. Menghilangkan redundansi data, memudahkan update, dan menjaga konsistensi
- C. Membuat database lebih lambat
- D. Membuat primary key menjadi otomatis

Esai:

1. Jelaskan ketiga level normalisasi (1NF, 2NF, 3NF) dengan menggunakan satu contoh kasus yang sama dari awal hingga akhir!
2. Apakah normalisasi penuh selalu yang terbaik? Kapan kita justru perlu melakukan denormalisasi?

29. WINDOW FUNCTION — Analitik Lanjutan

■ Penjelasan Mendalam

Window Function bekerja pada sekumpulan baris yang berhubungan (window) tanpa mengelompokkannya seperti GROUP BY. Setiap baris tetap ada, tapi mendapatkan nilai hasil kalkulasi dari seluruh window.

■ Contoh Query

ROW_NUMBER — penomoran baris

```
SELECT nama, umur,
ROW_NUMBER() OVER (ORDER BY umur ASC) AS nomor_urut
FROM users;
```

RANK vs DENSE_RANK

```
SELECT nama, umur,
RANK() OVER (ORDER BY umur DESC) AS rank_biasa,
DENSE_RANK() OVER (ORDER BY umur DESC) AS rank_dense
FROM users;
-- RANK ada gap jika nilai sama, DENSE_RANK tidak ada gap
```

SUM dengan PARTITION (per grup)

```
SELECT nama, kota, umur,
SUM(umur) OVER (PARTITION BY kota) AS total_umur_per_kota,
AVG(umur) OVER (PARTITION BY kota) AS rata_umur_per_kota
FROM users;
```

■ LATIHAN

Pilihan Ganda:

1. Apa perbedaan utama antara Window Function dan GROUP BY?
 - A. Tidak ada perbedaan
 - B. GROUP BY mengelompokkan baris menjadi satu hasil per grup, Window Function menambahkan kolom kalkulasi tanpa mengurangi jumlah baris
 - C. Window Function hanya bisa digunakan dengan ROW_NUMBER
 - D. GROUP BY lebih cepat
2. Apa perbedaan antara RANK() dan DENSE_RANK()?
 - A. Tidak ada perbedaan

- B. RANK() menghasilkan gap pada ranking jika ada nilai sama, DENSE_RANK() tidak
- C. DENSE_RANK() lebih lambat
- D. RANK() hanya untuk angka

3. Apa fungsi klausa PARTITION BY dalam Window Function?

- A. Mengurutkan baris dalam window
- B. Membagi data menjadi partisi untuk menerapkan window function secara terpisah per grup
- C. Membatasi jumlah baris
- D. Menyaring baris sebelum window function dijalankan

Esai:

1. Jelaskan kapan Window Function jauh lebih tepat dibandingkan kombinasi GROUP BY dan subquery!
2. Jelaskan kegunaan fungsi LAG() dan LEAD() dalam analisis data! Berikan contoh query!

30. CTE (Common Table Expression) — Query Lebih Terstruktur

■ Penjelasan Mendalam

CTE adalah cara mendefinisikan **query sementara yang diberi nama** menggunakan klausa **WITH**. CTE membuat query kompleks lebih mudah dibaca dengan memecahnya menjadi langkah-langkah logis. CTE hanya hidup selama satu eksekusi query.

■ Contoh Query

CTE dasar

```
WITH user_dewasa AS (  
  SELECT * FROM users WHERE umur > 22  
)  
SELECT * FROM user_dewasa ORDER BY umur DESC;  
  
-- Lebih mudah dibaca dibanding nested subquery!
```

CTE + Window Function

```
WITH ranked_users AS (  
  SELECT nama, kota, umur,  
    RANK() OVER (PARTITION BY kota ORDER BY umur DESC) AS rank_per_kota  
  FROM users  
)  
SELECT * FROM ranked_users WHERE rank_per_kota = 1;  
  
-- User tertua di setiap kota
```

■ LATIHAN

Pilihan Ganda:

1. Apa perbedaan utama antara CTE dan VIEW?
 - A. CTE lebih cepat dari VIEW
 - B. CTE bersifat sementara (hanya untuk satu query), VIEW disimpan permanen di database
 - C. VIEW bisa rekursif, CTE tidak
 - D. CTE hanya bisa digunakan sekali per query
2. Apa kata kunci yang digunakan untuk mendefinisikan CTE?
 - A. DEFINE

- B. TEMP
- C. WITH
- D. AS TABLE

3. Keuntungan utama CTE dibandingkan nested subquery?

- A. CTE selalu lebih cepat
- B. CTE membuat query lebih mudah dibaca dan dipelihara karena setiap langkah diberi nama deskriptif
- C. CTE bisa menyimpan data permanen
- D. CTE otomatis membuat index

Esai:

1. Tuliskan ulang query berikut menggunakan CTE: `SELECT * FROM (SELECT nama, umur, RANK() OVER (ORDER BY umur DESC) AS rank_umur FROM users) AS sub WHERE sub.rank_umur <= 2;`
2. Jelaskan konsep Recursive CTE! Untuk kasus data seperti apa Recursive CTE sangat berguna?

31. STORED PROCEDURE & FUNCTION — Logika di Database

■ Penjelasan Mendalam

Stored Procedure adalah sekumpulan perintah SQL yang disimpan di database dan bisa dipanggil berulang. **Function** mirip tapi selalu mengembalikan nilai dan bisa digunakan di SELECT.

■ Contoh Query

Function kategorisasi umur (PostgreSQL)

```
CREATE OR REPLACE FUNCTION kategori_umur(usia INT)
RETURNS VARCHAR AS $$
BEGIN
IF usia < 21 THEN RETURN 'Remaja';
ELSIF usia <= 25 THEN RETURN 'Dewasa Muda';
ELSE RETURN 'Dewasa';
END IF;
END;
$$ LANGUAGE plpgsql;

-- Penggunaan:
SELECT nama, umur, kategori_umur(umur) AS kategori FROM users;
```

Function total pembelian

```
CREATE OR REPLACE FUNCTION total_pembelian(uid INT)
RETURNS DECIMAL AS $$
SELECT COALESCE(SUM(harga), 0) FROM orders WHERE user_id = uid;
$$ LANGUAGE sql;

-- Penggunaan:
SELECT nama, total_pembelian(id) AS total_belanja FROM users;
```

■ LATIHAN

Pilihan Ganda:

1. Apa perbedaan utama antara Function dan Stored Procedure?

- A. Function lebih lambat
- B. Function wajib mengembalikan nilai dan bisa digunakan di SELECT; Stored Procedure tidak wajib return nilai
- C. Stored Procedure bisa di-cache, Function tidak
- D. Tidak ada perbedaan fungsional

2. Apa keuntungan menggunakan Function di database?

- A. Function di database selalu lebih cepat
- B. Logika tersedia untuk semua aplikasi yang terhubung dan mengurangi traffic data
- C. Function di database lebih mudah di-debug
- D. Function tidak membutuhkan testing

3. COALESCE(nilai, 0) dalam SQL berfungsi untuk?

- A. Menggabungkan dua nilai
- B. Mengembalikan nilai pertama yang tidak NULL; jika semua NULL, kembalikan nilai default
- C. Menghitung jumlah nilai
- D. Mengkonversi tipe data

Esai:

1. Jelaskan kapan sebaiknya logika bisnis ditempatkan di database vs di kode aplikasi!
2. Tuliskan sebuah function PostgreSQL yang menerima user_id dan mengembalikan ringkasan user sebagai teks!

32. TRIGGER — Otomasi di Database

■ Penjelasan Mendalam

TRIGGER adalah prosedur yang **otomatis dieksekusi** sebagai respons terhadap event (INSERT, UPDATE, DELETE) pada sebuah tabel. Bisa diatur BEFORE atau AFTER event.

■ Contoh Query

Auto-update timestamp

```
CREATE OR REPLACE FUNCTION set_updated_at()  
RETURNS TRIGGER AS $$  
BEGIN  
    NEW.updated_at = CURRENT_TIMESTAMP;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trigger_updated_at  
BEFORE UPDATE ON users  
FOR EACH ROW  
EXECUTE FUNCTION set_updated_at();
```

Audit log perubahan kota

```
CREATE OR REPLACE FUNCTION log_kota_change()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF OLD.kota != NEW.kota THEN  
        INSERT INTO user_audit (user_id, action, old_kota, new_kota)
```

```
VALUES (OLD.id, 'UPDATE', OLD.kota, NEW.kota);
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

■ LATIHAN

Pilihan Ganda:

1. Kapan TRIGGER dieksekusi?
 - A. Hanya saat aplikasi memanggilnya secara manual
 - B. Secara otomatis oleh database sebagai respons terhadap event (INSERT/UPDATE/DELETE)
 - C. Setiap 1 jam secara terjadwal
 - D. Hanya saat database di-restart
2. Apa perbedaan antara TRIGGER BEFORE dan AFTER?
 - A. BEFORE lebih cepat dari AFTER
 - B. BEFORE dieksekusi sebelum operasi utama (bisa memodifikasi data), AFTER dieksekusi setelah operasi selesai
 - C. AFTER bisa membatalkan operasi, BEFORE tidak
 - D. Tidak ada perbedaan fungsional
3. Dalam trigger PostgreSQL, NEW dan OLD merujuk pada?
 - A. Versi terbaru dan terlama dari database
 - B. NEW adalah nilai baru yang akan disimpan, OLD adalah nilai sebelum perubahan
 - C. Trigger baru dan trigger lama
 - D. Tabel baru dan tabel lama

Esai:

1. Jelaskan bagaimana TRIGGER dapat digunakan untuk membuat sistem audit log yang lengkap!
2. Apa potensi masalah jika terlalu banyak trigger pada satu tabel? Jelaskan juga konsep cascading trigger!

33. QUERY OPTIMIZATION — Performa Query

■ Penjelasan Mendalam

Query optimization adalah seni menulis query SQL agar **dieksekusi seefisien mungkin**. Gunakan **EXPLAIN** dan **EXPLAIN ANALYZE** untuk melihat rencana eksekusi query.

■ Contoh Query

EXPLAIN untuk analisis query

```
-- Lihat rencana eksekusi (tanpa menjalankan):
EXPLAIN SELECT * FROM users WHERE nama = 'Budi';

-- Jalankan dan lihat statistik aktual:
EXPLAIN ANALYZE SELECT * FROM users WHERE nama = 'Budi';

-- Istilah: Seq Scan (baca semua baris), Index Scan (pakai index)
```

Best practices optimasi

```
-- ■ Hindari SELECT *:
```

```

SELECT * FROM users WHERE id = 1;

-- ■ Pilih kolom spesifik:
SELECT nama, email FROM users WHERE id = 1;

-- ■ Fungsi pada kolom WHERE (tidak bisa pakai index):
SELECT * FROM users WHERE LOWER(nama) = 'budi';

-- ■ Langsung:
SELECT * FROM users WHERE nama = 'Budi';

-- ■ Gunakan EXISTS untuk subquery besar:
SELECT * FROM users u
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);

```

■ LATIHAN

Pilihan Ganda:

- Perintah SQL apa yang digunakan untuk melihat rencana eksekusi sebuah query?
 - DESCRIBE query
 - SHOW PLAN query
 - EXPLAIN query
 - ANALYZE query
- Mengapa menggunakan fungsi pada kolom di WHERE (WHERE LOWER(nama)) bisa memperlambat query?
 - Fungsi LOWER membutuhkan banyak memori
 - Database tidak bisa menggunakan index karena harus menghitung fungsi untuk setiap baris
 - LOWER hanya bisa digunakan di SELECT
 - Tidak ada dampak performa
- Apa itu N+1 Query Problem?
 - Query yang mengambil N+1 kolom
 - Pola buruk di mana 1 query mengambil daftar, lalu N query terpisah dijalankan untuk setiap item — bisa digantikan dengan satu JOIN
 - Tabel yang memiliki N+1 index
 - Error karena terlalu banyak JOIN

Esai:

- Jelaskan apa itu Seq Scan dan Index Scan dalam EXPLAIN output! Kapan Seq Scan tidak selalu buruk?
- Tuliskan 5 best practice dalam menulis query SQL yang efisien dengan contoh query buruk dan versi perbaikannya!

■ LEVEL BASIC

1. SELECT

PG: 1. B | 2. C | 3. C

Esai 1: SELECT * mengambil semua kolom — boros bandwidth dan berpotensi ekspos kolom sensitif. SELECT nama, umur hanya ambil kolom yang dibutuhkan — lebih cepat dan aman.

Esai 2: SELECT nama, umur * 2 AS dua_kali_umur FROM users; — nama adalah kolom biasa, umur * 2 adalah kalkulasi, AS dua_kali_umur memberi nama hasil kalkulasi.

2. WHERE

PG: 1. C | 2. A | 3. D

Esai 1: AND: kedua kondisi harus terpenuhi. Contoh: WHERE umur > 20 AND kota = 'Bandung' → hanya Rina. OR: cukup satu kondisi. Contoh: WHERE kota = 'Bandung' OR kota = 'Jakarta' → Budi, Andi, Rina.

Esai 2: Tanpa WHERE, DELETE dan UPDATE berlaku pada SEMUA baris. Contoh: DELETE FROM users; menghapus seluruh data. UPDATE users SET kota = 'Jakarta'; memindahkan semua user ke Jakarta.

3. INSERT

PG: 1. B | 2. C | 3. C

Esai 1: INSERT banyak baris hanya butuh satu round-trip ke database vs N round-trips untuk N INSERT terpisah. Mengurangi overhead koneksi dan meningkatkan throughput signifikan.

Esai 2: NULL berarti nilai tidak diketahui/tidak ada. Diperbolehkan jika kolom tidak memiliki NOT NULL. Tidak diperbolehkan jika kolom memiliki NOT NULL atau merupakan PRIMARY KEY.

4. UPDATE

PG: 1. C | 2. B | 3. C

Esai 1: Tanpa WHERE, semua baris terpengaruh. Di e-commerce: UPDATE products SET harga = 0; tanpa WHERE akan mengubah harga semua produk menjadi gratis.

Esai 2: UPDATE users SET umur = umur + 1 WHERE kota = 'Bandung'; — Gunakan umur + 1 karena nilai umur masing-masing berbeda; kita ingin menambah 1 dari nilai yang sudah ada.

5. DELETE

PG: 1. B | 2. C | 3. C

Esai 1: DELETE: hapus baris tertentu, bisa di-rollback dalam transaksi. TRUNCATE: hapus semua baris lebih cepat, reset auto-increment. DROP: hapus seluruh tabel + struktur, tidak bisa di-undo.

Esai 2: Pencegahan: jalankan SELECT dulu sebelum DELETE; gunakan TRANSACTION; batasi hak akses; backup rutin. Pemulihan: gunakan backup terakhir atau point-in-time recovery.

6. ORDER BY

PG: 1. B | 2. C | 3. B

Esai 1: Di e-commerce: ORDER BY kategori ASC, harga ASC — urutkan per kategori (A-Z) dan dalam satu kategori dari termurah. User melihat produk terurut rapi.

Esai 2: ORDER BY tidak mengubah data di database. Data tetap tersimpan dalam urutan fisik aslinya. ORDER BY hanya mempengaruhi urutan tampilan hasil yang dikembalikan untuk satu query.

7. LIMIT & OFFSET

PG: 1. B | 2. B | 3. C

Esai 1: Formula: OFFSET = (halaman - 1) × per_halaman. Halaman 1: LIMIT 10 OFFSET 0. Halaman 2: LIMIT 10 OFFSET 10. Halaman 5: LIMIT 10 OFFSET 40.

Esai 2: OFFSET besar sangat lambat karena database tetap harus membaca dan membuang semua baris sebelum OFFSET. Alternatif: keyset pagination — WHERE id > last_seen_id LIMIT 10.

■ LEVEL INTERMEDIATE

8. COUNT

PG: 1. B | 2. C | 3. C

Esai 1: (1) Dashboard: 'Total user: 10.432'. (2) Cek duplikat sebelum INSERT: `SELECT COUNT(*) FROM users WHERE email = 'test@mail.com'`.

Esai 2: `COUNT(kolom)` mengabaikan NULL; `COUNT(*)` hitung semua baris. Jika 3 dari 4 user mengisi umur: `COUNT(umur) = 3`, `COUNT(*) = 4`.

9. GROUP BY

PG: 1. B | 2. B | 3. B

Esai 1: Database tidak tahu nilai nama mana yang harus ditampilkan untuk grup 'Bandung' (Budi atau Rina?). Hasilnya error atau tidak deterministik.

Esai 2: `SELECT kota, COUNT(*) AS jumlah, AVG(umur) AS rata_umur FROM users GROUP BY kota ORDER BY jumlah DESC`;

10. CREATE TABLE

PG: 1. C | 2. B | 3. B

Esai 1: `VARCHAR(100)`: batas 100 karakter, gunakan untuk nama/email. `TEXT`: panjang tak terbatas, untuk deskripsi/konten panjang.

Esai 2: `CREATE TABLE students (id SERIAL PK, nim VARCHAR(20) NOT NULL UNIQUE, nama VARCHAR(150) NOT NULL, jurusan VARCHAR(100) NOT NULL, angkatan INT NOT NULL, ipk DECIMAL(3,2) DEFAULT 0.00)`;

11. PRIMARY KEY

PG: 1. C | 2. B | 3. C

Esai 1: Natural Key (NIK, email): risiko berubah. Surrogate Key (integer/UUID buatan): stabil, pendek. Surrogate Key lebih direkomendasikan.

Esai 2: Composite PK ketika tidak ada satu kolom yang bisa jadi identitas unik sendiri. Contoh: tabel enrollments dengan (student_id, course_id) — keduanya baru unik sebagai kombinasi.

12. DISTINCT

PG: 1. B | 2. C | 3. C

Esai 1: `DISTINCT` lebih singkat untuk sekadar mendapat nilai unik. `GROUP BY` lebih fleksibel karena bisa digabung dengan agregat. Gunakan `DISTINCT` untuk kesederhanaan.

Esai 2: `SELECT DISTINCT brand FROM products ORDER BY brand`; — berguna untuk dropdown filter di halaman pencarian e-commerce.

13. LIKE

PG: 1. B | 2. C | 3. B

Esai 1: `%` = nol atau lebih karakter: `'B%'` cocok Budi, Besar, B. `_` = tepat satu karakter: `'B_di'` hanya cocok Budi.

Esai 2: `LIKE '%kata%'` tidak bisa pakai index B-tree biasa karena harus scan semua nilai dari awal. Alternatif: Full-Text Search atau Elasticsearch.

14. IN

PG: 1. B | 2. B | 3. B

Esai 1: `WHERE kota IN ('Bandung','Jakarta','Surabaya')` jauh lebih ringkas dari 3 kondisi OR, mudah ditambah/dikurangi nilai.

Esai 2: Jika salah satu nilai `NOT IN` adalah NULL, hasilnya selalu kosong! SQL tidak bisa menentukan apakah nilai 'tidak sama dengan NULL'. Solusi: gunakan `NOT EXISTS`.

15. BETWEEN

PG: 1. B | 2. C | 3. B

Esai 1: `WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31'` — inklusif, termasuk 1 Januari dan 31 Desember. Umum untuk laporan bulanan/tahunan.

Esai 2: `BETWEEN` bersifat inklusif artinya kedua batas termasuk dalam rentang. `BETWEEN 20 AND 25` identik dengan `>= 20 AND <= 25`.

16. ALIAS

PG: 1. B | 2. B | 3. B

Esai 1: Tanpa alias: `SELECT users.nama, orders.produk FROM users JOIN orders ON users.id = orders.user_id`; — dengan alias: `SELECT u.nama, o.produk FROM users u JOIN orders o ON u.id = o.user_id`;

Esai 2: Tidak boleh pakai alias kolom di `WHERE` karena urutan eksekusi: `FROM`→`WHERE`→`GROUP BY`→`HAVING`→`SELECT`. Alias didefinisikan di `SELECT`, tapi `WHERE` diproses sebelum `SELECT`.

17. SUM & AVG

PG: 1. B | 2. C | 3. B

Esai 1: $AVG(kolom) = \frac{SUM(tidak\ NULL)}{COUNT(tidak\ NULL)}$. $\frac{SUM}{COUNT(*)} = \frac{SUM}{COUNT(semua\ baris)}$. Jika ada NULL, $COUNT(*)$ lebih besar $\rightarrow \frac{SUM}{COUNT(*)}$ lebih kecil dari AVG.

Esai 2: `SELECT MIN(harga) AS terkecil, MAX(harga) AS terbesar, SUM(harga) AS total, AVG(harga) AS rata FROM orders;`

18. HAVING

PG: 1. B | 2. B | 3. B

Esai 1: WHERE menyaring bahan mentah sebelum dimasak. GROUP BY memasak per porsi. HAVING memilih porsi yang layak disajikan.

Esai 2: `SELECT kota, AVG(umur) AS rata_umur FROM users GROUP BY kota HAVING AVG(umur) > 20 ORDER BY rata_umur DESC;`

19. JOIN

PG: 1. B | 2. B | 3. B

Esai 1: LEFT JOIN berguna saat ingin melihat semua data termasuk yang belum punya data terkait. Contoh: semua user + pesannya — user belum pernah pesan tetap muncul dengan NULL.

Esai 2: Keuntungan normalisasi: hemat storage, update konsisten di satu tempat, integritas data, tabel lebih kecil $\rightarrow$ query lebih cepat.

20. SUBQUERY

PG: 1. B | 2. C | 3. B

Esai 1: `SELECT * FROM users WHERE umur > (SELECT AVG(umur) FROM users);` — atau dengan CTE: `WITH avg_umur AS (SELECT AVG(umur) AS avg FROM users) SELECT u.* FROM users u, avg_umur a WHERE u.umur > a.avg;`

Esai 2: Correlated subquery mereferensikan kolom dari query luar — dieksekusi satu kali per baris (lambat). Subquery biasa dieksekusi sekali. Contoh: `umur > (SELECT AVG(umur) FROM users WHERE kota = u.kota)`.

21. INDEX

PG: 1. B | 2. B | 3. B

Esai 1: Full Table Scan: baca setiap baris dari awal. Pada 10 juta baris = 10 juta pembacaan. Dengan index: langsung lompat ke data menggunakan B-tree — $O(\log n)$ vs $O(n)$.

Esai 2: (1) Kolom jarang dipakai di WHERE/JOIN. (2) Kolom dengan sedikit variasi nilai (boolean). (3) Tabel kecil. (4) Kolom yang sangat sering di-UPDATE.

■ LEVEL ADVANCED

22. UNION & UNION ALL

PG: 1. B | 2. B | 3. B

Esai 1: UNION melakukan deduplication: sort semua data, bandingkan baris, hapus duplikat — $O(n \log n)$. UNION ALL langsung gabungkan tanpa sorting.

Esai 2: Gabungkan active_users dan archived_users: `SELECT nama, email, 'active' AS status FROM active_users UNION SELECT nama, email, 'archived' AS status FROM archived_users;`

23. CASE

PG: 1. C | 2. C | 3. B

Esai 1: `SELECT u.nama, o.produk, o.harga, CASE WHEN o.harga < 500000 THEN 'Budget' WHEN o.harga BETWEEN 500000 AND 5000000 THEN 'Mid-range' ELSE 'Premium' END AS kategori FROM users u JOIN orders o ON u.id = o.user_id;`

Esai 2: `SELECT COUNT(CASE WHEN umur < 21 THEN 1 END) AS remaja, COUNT(CASE WHEN umur BETWEEN 21 AND 25 THEN 1 END) AS dewasa_muda, COUNT(CASE WHEN umur > 25 THEN 1 END) AS dewasa FROM users;`

24. VIEW

PG: 1. B | 2. B | 3. C

Esai 1: Regular View: query dieksekusi ulang setiap kali diakses — selalu data terbaru tapi bisa lambat. Materialized View: hasil tersimpan fisik, lebih cepat tapi bisa tidak real-time.

Esai 2: Buat view public_users: `CREATE VIEW public_users AS SELECT id, nama, kota FROM users;` — berikan akses ke view ini, bukan ke tabel users yang punya kolom sensitif.

25. ALTER TABLE

PG: 1. B | 2. C | 3. B

Esai 1: Strategi aman: (1) Tambah kolom baru. (2) Isi dari kolom lama. (3) Update aplikasi. (4) Hapus kolom lama setelah yakin.

Esai 2: MySQL: CHANGE COLUMN old_name new_name data_type. PostgreSQL: RENAME COLUMN old TO new. Penting untuk migrasi antar database.

26. NOT NULL & DEFAULT

PG: 1. B | 2. C | 3. B

Esai 1: Bug dari NULL: AVG(umur) hanya hitung user yang mengisi umur — hasilnya menyesatkan jika banyak NULL. WHERE umur = NULL tidak pernah menemukan apapun (harus IS NULL).

Esai 2: CREATE TABLE payments (id SERIAL PK, user_id INT NOT NULL, amount DECIMAL(15,2) NOT NULL, status VARCHAR(20) NOT NULL DEFAULT 'pending', payment_method VARCHAR(50), paid_at TIMESTAMP, created_at TIMESTAMP DEFAULT NOW());

27. TRANSACTION

PG: 1. B | 2. B | 3. B

Esai 1: A (Atomicity): semua atau tidak sama sekali. C (Consistency): data selalu valid. I (Isolation): transaksi tidak saling mengganggu. D (Durability): data COMMIT tidak hilang meski crash.

Esai 2: Pemesanan tiket: BEGIN; UPDATE seats SET is_booked = TRUE WHERE seat_id = 5; INSERT INTO bookings (user_id, seat_id) VALUES (1, 5); COMMIT; — jika salah satu gagal, ROLLBACK semua.

28. NORMALISASI

PG: 1. C | 2. B | 3. B

Esai 1: 1NF: setiap sel satu nilai atomic. 2NF: hilangkan partial dependency (non-key bergantung pada sebagian PK). 3NF: hilangkan transitive dependency (non-key bergantung pada non-key lain).

Esai 2: Normalisasi tidak selalu terbaik. Denormalisasi dilakukan ketika performa kritis dan JOIN terlalu lambat, atau untuk OLAP/data warehouse. Risiko: duplikasi, inkonsistensi.

29. WINDOW FUNCTION

PG: 1. B | 2. B | 3. B

Esai 1: WITH ranked AS (SELECT *, RANK() OVER (PARTITION BY kategori ORDER BY total_terjual DESC) AS rk FROM products) SELECT * FROM ranked WHERE rk <= 3;

Esai 2: LAG() ambil nilai baris sebelumnya — bandingkan dengan periode sebelumnya. LEAD() ambil nilai baris berikutnya. Contoh: LAG(umur) OVER (ORDER BY umur) AS umur_sebelumnya.

30. CTE

PG: 1. B | 2. C | 3. B

Esai 1: WITH ranked_users AS (SELECT nama, umur, RANK() OVER (ORDER BY umur DESC) AS rank_umur FROM users) SELECT * FROM ranked_users WHERE rank_umur <= 2; — lebih mudah dibaca karena setiap langkah bernama.

Esai 2: Recursive CTE untuk data hierarkis: WITH RECURSIVE subordinates AS (SELECT * FROM employees WHERE id = 1 UNION ALL SELECT e.* FROM employees e JOIN subordinates s ON e.manager_id = s.id) SELECT * FROM subordinates;

31. STORED PROCEDURE & FUNCTION

PG: 1. B | 2. B | 3. B

Esai 1: Di DB: logika tersedia untuk semua aplikasi, aman (user tidak akses tabel langsung). Di aplikasi: mudah version control, portable. Rekomendasi: logika bisnis di aplikasi.

Esai 2: CREATE OR REPLACE FUNCTION ringkasan_user(uid INT) RETURNS TEXT AS \$\$ DECLARE r TEXT; BEGIN SELECT CONCAT('Nama: ', nama, ', Kota: ', kota, ', Total: ', COALESCE(SUM(o.harga),0)) INTO r FROM users u LEFT JOIN orders o ON u.id = o.user_id WHERE u.id = uid GROUP BY u.nama, u.kota; RETURN r; END; \$\$ LANGUAGE plpgsql;

32. TRIGGER

PG: 1. B | 2. B | 3. B

Esai 1: Audit log minimum: id, tabel_name, record_id, action (INSERT/UPDATE/DELETE), old_values (JSONB), new_values (JSONB), changed_by, changed_at. Penting untuk compliance dan forensik.

Esai 2: Terlalu banyak trigger: memperlambat write, urutan tidak terduga, sulit di-debug. Cascading trigger bisa menyebabkan loop tak terbatas. Solusi: dokumentasikan dan batasi penggunaan.

33. QUERY OPTIMIZATION

PG: 1. C | 2. B | 3. B

Esai 1: Seq Scan: baca setiap baris dari awal (seperti baca buku dari hal. 1). Index Scan: langsung lompat ke data. Seq Scan tidak selalu buruk: untuk tabel kecil bisa lebih cepat dari overhead index.

Esai 2: (1) Pilih kolom spesifik, hindari SELECT *. (2) Pastikan kolom WHERE ber-index. (3) Hindari fungsi pada kolom WHERE. (4) Gunakan JOIN bukan subquery. (5) Gunakan LIMIT untuk data besar.

■ Modul Lengkap Belajar SQL · Dari Dasar Hingga Advanced · 33 Topik · Basic | Intermediate | Advanced